

Prompting or Fine-tuning? A Comparative Study of Large Language Models for Taxonomy Construction

Boqi Chen^{*} 

Electrical and Computer Engineering
McGill University
Montreal, Canada

Fandi Yi^{*} 

Desautels Faculty of Management
McGill University
Montreal, Canada

Dániel Varró 

Linköping University
McGill University
Linköping, Sweden / Montreal, Canada

Abstract—Taxonomies represent hierarchical relations between entities, frequently applied in various software modeling and natural language processing (NLP) activities. They are typically subject to a set of structural constraints restricting their content. However, manual taxonomy construction can be time-consuming, incomplete, and costly to maintain. Recent studies of large language models (LLMs) have demonstrated that appropriate user inputs (called *prompting*) can effectively guide LLMs, such as GPT-3, in diverse NLP tasks without explicit (re-)training. However, existing approaches for automated taxonomy construction typically involve *fine-tuning* a language model by adjusting model parameters. In this paper, we present a general framework for taxonomy construction that takes into account structural constraints. We subsequently conduct a systematic comparison between the prompting and fine-tuning approaches performed on a hypernym taxonomy and a novel computer science taxonomy dataset. Our result reveals the following: (1) Even without explicit training on the dataset, the prompting approach outperforms fine-tuning-based approaches. Moreover, the performance gap between prompting and fine-tuning widens when the training dataset is small. However, (2) taxonomies generated by the fine-tuning approach can be easily post-processed to satisfy all the constraints, whereas handling violations of the taxonomies produced by the prompting approach can be challenging. These evaluation findings provide guidance on selecting the appropriate method for taxonomy construction and highlight potential enhancements for both approaches.

Index Terms—taxonomy construction, domain-specific constraints, large language models, few-shot learning, fine-tuning

I. INTRODUCTION

Context: Taxonomies are models that describe hierarchical relations between concepts or entities within a particular domain. The structure of a taxonomy is commonly limited by some taxonomy-specific constraints [1]. Hierarchical relations have significant importance in software engineering practice, including composition or generalization relations in domain modeling, hierarchical models in database systems, and inheritance in most object-oriented languages. Additionally, tax-

onomies are extensively applied in semantic web applications, typically in the form of OWL ontologies [2].

Nevertheless, manually constructing taxonomies is typically time-consuming, prone to be incomplete, and challenging to maintain. For example, the relations in the WordNet taxonomy have been demonstrated to be incomplete [3]. To address the issues associated with manual construction, many approaches aim to automate taxonomy construction from a set of concepts [3]–[5]. However, these methods often focus on hypernym taxonomies utilizing widely available datasets, while their performance in other domains lacking such datasets remains less explored. Furthermore, although existing approaches incorporate structural constraints for taxonomies [4], [5], they rely on tree structures within the hypernym taxonomy and lack generalizability to other types of constraints.

Problem statement: The recent advances in generative large language models (LLMs) spark growing interest in their application to diverse software engineering activities, offering the potential for tackling classic software engineering challenges with novel approaches. Several powerful LLMs have demonstrated remarkable performance across various tasks relying solely on instructive input, without substantial task-specific training data. Consequently, an important question arises: in scenarios where some training data is available, is it still necessary to train a specialized model, or would instructive input prompts provided to a generative LLM be sufficient?

Our paper aims to address this question for automated taxonomy construction. Given a set of concepts with semantically meaningful names, the goal is to derive a taxonomy that captures the hierarchical relations among these concepts while fulfilling the *structural constraints*.

Objective: This paper presents a comparative study using LLMs for taxonomy construction with datasets from two domains. Specifically, we compare two approaches: (1) a widely adapted approach that trains an LLM on a dedicated training set for taxonomy construction (referred to as “*fine-tuning*”) and (2) a recent approach that provides instructive input to guide powerful generative LLMs (referred to as “*prompting*”). Our objective is to identify the strengths and limitations of both approaches and discuss potential improvements.

^{*} Equal contribution

Partially supported by the FRQNT-B2X project (file number: 319955) and the Wallenberg AI, Autonomous Systems and Software Program (WASP), Sweden

Contribution: Given a set of concepts, we propose a general framework to generate a taxonomy using LLMs. We subsequently compare two taxonomy construction methods within this general framework. The specific contributions of this paper can be summarized as follows.

- We propose a general framework for taxonomy construction, which builds upon existing approaches to generate taxonomies from LLM-based relation predictions.
- We formulate the taxonomy generation problem as a text generation task and propose a few-shot prompt for constructing taxonomies without explicit training.
- We provide a new dataset specific to the computer science domain derived from the hierarchical relations within the ACM Computing Classification System (CCS).
- We conduct experiments using two fine-tuning approaches and a novel prompting approach for taxonomy construction, using two distinct taxonomy datasets. The quality of the generated taxonomies is evaluated against the ground truth, along with their structure consistency regarding the constraints.

Added value: To the best of our knowledge, this paper presents the first use of generative LLMs for taxonomy construction without explicit training. We present a comprehensive comparison of this approach against fine-tuning approaches. Furthermore, we extend the scope of our experiments beyond commonly adapted hypernym taxonomy by incorporating a novel domain-specific taxonomy focusing on computer science concepts. In our evaluation, we also assess both the performance of the generated taxonomies against the ground truth and their structure consistency. This paper provides valuable insights into leveraging LLMs for prediction hierarchical relations and demonstrates the impact of consistency in enhancing the performance of machine learning components.

II. BACKGROUND

A. Taxonomy Construction

Given a set of concepts, taxonomy construction involves identifying the hierarchical relations between these concepts, such as parent-child or inclusion relations. Formally, a taxonomy is defined as a *directed* graph $T = (\mathcal{N}, \mathcal{E})$, where $\mathcal{N}$ represents the set of concepts (nodes) and $\mathcal{E}$ represents hierarchical relations (edges) between concepts where $\overrightarrow{AB} \in \mathcal{E}$ denotes that concept A is a parent of concept B . Taxonomies are typically subject to specific structural constraints imposed by the domain [1]. Typically, a taxonomy is always *connected* and *acyclic*. Depending on whether a concept in the taxonomy can have multiple parents, the taxonomy may form a tree (for uni-parent) or DAG (for multi-parent). Relations in a taxonomy may also be transitive: if A is a parent of B , B is a parent of C , then A is also a parent of C .

A taxonomy is considered to *consistent* if it satisfies these constraints. The constraints are denoted as $\mathcal{C}$. The task of *taxonomy construction* aims to create a taxonomy based on a set of concepts while aiming to satisfy these constraints [3].

Example 1: Figure 1 illustrates an example of taxonomy construction. Given the set of five terms on the left, taxonomy

construction generates a hierarchical graph of these terms, as shown on the right. The edges in the graph represent hierarchical relations, e.g., “learning paradigms” is a parent of “supervised learning”.

This paper focuses on the relation prediction-based taxonomy construction, as described by Chen et al. [5]. This setup involves the initial prediction of relations for each concept pair, followed by the construction of output taxonomy through a post-processing procedure. Formally, given a set of concepts denoted as $\mathcal{N}$ and taxonomy constraints represented by $\mathcal{C}$, the relation prediction process begins by assigning a score $s_{(A,B)}$ to each pair of concepts $A, B \in \mathcal{N}, A \neq B$. The target is then to create a taxonomy $T = (\mathcal{N}, \mathcal{E})$ that maximizes the sum of the edges scores while adhering to the specified constraints.

B. Taxonomy Relations Applications

Taxonomy relations exist in many knowledge engineering and modeling applications. The most common application in knowledge engineering is the *hypernym* relations in lexicon databases such as WordNet [6], while the taxonomy relations form a tree of related words. Inheritance (or generalization) is also a type of taxonomy relation and typically exists in most object-oriented languages, domain modeling, relational databases, and OWL ontologies. Depending on whether multi-parent is allowed, the relations can either form a tree or DAG. Finally, aggregation and composition relations in UML class diagrams can also be seen as tree-structured taxonomies.

C. Language Models

a) *Large language models (LLMs):* Language modeling is a classic task in natural language processing (NLP) that involves predicting the probability of a sequence of tokens. Large language models (LLMs) leverage deep neural networks, commonly using transformer blocks [7].

For a sequence of input tokens (referred to as a *prompt*) $s = \{s_1, s_2, \dots, s_{k-1}\}$, LLMs estimate the probability of the next token $P(s_k | s_1, \dots, s_{k-1})$. LLMs can also be used for text generation through auto-regression, where the newly predicted token is appended to the token sequence to predict subsequent tokens. In this scenario, a search method such as beam search is typically utilized to generate the final output sequence [8]. However, such search methods also introduce variability to the output of LLMs across different runs for the same input.

After pre-training an LLM for language modeling, it can be customized for other tasks by fine-tuning, which involves updating a subset of its parameters [9]. Recently, LLMs such as GPT-3 [8] and GPT-4 [10] show remarkable generalizability, as they can be adapted to other tasks by providing a few examples within the prompt, without parameter updates [8].

This paper explores the application of LLM fine-tuning and prompting techniques in the context of taxonomy construction. In particular, we compare fine-tuning approaches for widely-used open-source and cost-effective LLMs, while also investigating the potential of prompting techniques for more powerful, black-box LLMs accessible through an API.

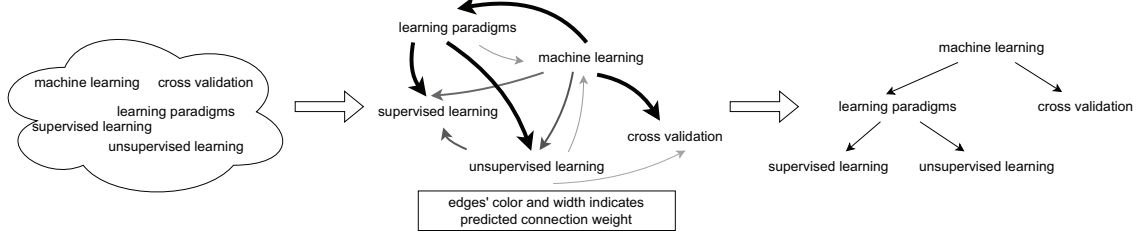


Fig. 1: An example of taxonomy construction

b) Fine-tuning: Fine-tuning of an LLM involves initially *pre-training* the LLM with a large corpus for language modeling and subsequently adapting it to a different task by training a subset of the LLM's parameters using a task-specific dataset. In this paper, we explore two widely-used fine-tuning methods for LLMs: the *layer-wise* approach and *LoRA*.

Layer-wise approach is a classic fine-tuning method that focuses on updating only a subset of parameters in an LLM. For a pre-trained autoregressive LLM with l layers, parameters from the last z layers are updated and the initial $l-z$ layers are fixed during training for the task. This approach leverages the fixed layers of the LLM to capture task-independent high-level information from the input. Subsequently, this information is used by the fine-tuned layers to create task-specific output.

LoRA (low rank adaptation) for LLMs is a fine-tuning approach that significantly reduces the number of parameters requiring updates in training [11]. This method treats the fine-tuning of each weight matrix $W \in \mathbb{R}^{(m,n)}$ as the learning of a delta matrix ΔW enabling the representation fine-tuned matrix W' as the sum of the two matrices: $W' = W + \Delta W$. In many cases, ΔW exhibits low-rank characteristics [12], [13], indicating that it can be decomposed as the product of two smaller matrices $\Delta W = A \times B$, such that $A \in \mathbb{R}^{(m,d)}$, $B \in \mathbb{R}^{(d,n)}$. Typically, the value of d is chosen so that the parameter count of matrices A and B is notably smaller than that of matrix W : $m \times d + d \times n \ll m \times n$. The adoption of LoRA can lead to significant reductions in memory overhead and storage costs during training, making it suitable for taxonomy construction that requires different models across different domains.

c) Prompting: Instead of updating the weights of the LLM, in case of prompting, users provide instructions and examples as input of the LLM in order to guide its output towards the desired task. We focus on one popular prompting method known as *few-shot prompting*.

Few-shot prompting aims to teach the LLM how to adapt to a specific task by carefully constructing an instruction that includes a few examples extracted from the task dataset. For example, in taxonomy construction, one can begin by providing instruction on how the taxonomy should be generated, followed by a list of example taxonomies, and concluding with the set of concept names as input to the taxonomy construction. It has been demonstrated that few-shot prompting can yield remarkable results, particularly when implemented with large-scale LLMs consisting of hundreds to thousands of

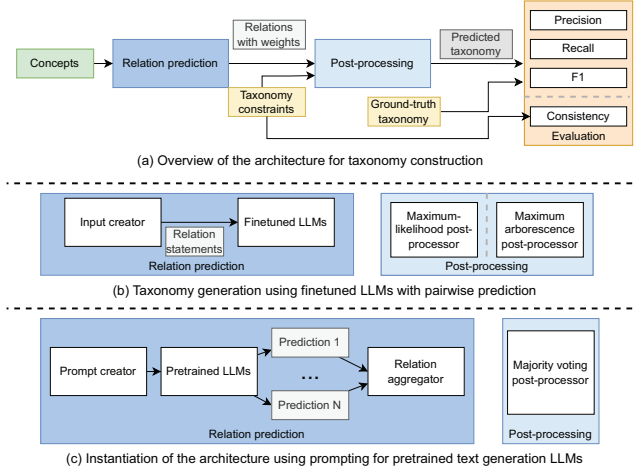


Fig. 2: Architecture for taxonomy generation using LLMs. Common components are omitted for (b) and (c).

billions of parameters [8]. However, the effectiveness of this approach diminishes when applied to less powerful LLMs.

III. APPROACH

A. Overview

a) Problem formulation: Given a set of concepts and constraints, *taxonomy construction* is to create a taxonomy that accurately represents the hierarchical structure among these concepts based on their names. Formally, let $\mathcal{N}$ denote a set of concepts, $\mathcal{C}$ represent the constraints for the taxonomy, and T be the ground-truth taxonomy of these concepts. The objective of taxonomy construction is to establish a mapping function $\mathfrak{f}$ such that the generated taxonomy T' is obtained as $T' = \mathfrak{f}(\mathcal{N}, \mathcal{C})$. The generated taxonomy T' should be consistent with the constraints in $\mathcal{C}$ and *similar* to the ground truth taxonomy T . Ideally, they should be identical: $T' \cong T$. In this paper, our objective is to evaluate the capability of LLMs in the context of $\mathfrak{f}$, considering both the fine-tuning and prompting approaches.

The structural constraints governing a graph can often be expressed using first-order logic or high-level constraint languages [14]. For the sake of conciseness, we provide a textual description of these constraints. In this paper, our focus

is on taxonomies that have a tree structure [1]. Therefore, we define the set of constraints $\mathcal{C}$ as follows.

- 1) **Uniqueness of the root:** There exists exactly one node that does not have a parent node in each taxonomy.
- 2) **Uniqueness of the parent:** For all non-root nodes, exactly one parent node exists.

b) *Architecture:* The general architecture of our taxonomy generation approach is illustrated in the top diagram of Figure 2. The construction of a taxonomy involves two main steps. Firstly, the *relation prediction* component generates a set of candidate relations along with their associated weights. These weighted relations are then subjected to a *post-processing* stage, which can take into consideration the imposed constraints to generate the final taxonomy. Finally, the predicted taxonomies are evaluated using the ground-truth taxonomy and metrics, which will be described in Section V.

In the remainder of this section, we first discuss the general architecture in detail. Next, we explore two LLM-based approaches using the general architecture: fine-tuning (Figure 2(b)) and prompting (Figure 2(c)). These two approaches mainly differ in the generation of candidate relations and the post-processing of these relations.

B. General architecture

a) *Relation prediction:* The *relation prediction* component aims to generate a set of candidate relations and their corresponding weights based on *input concepts*. This component typically relies on a machine learning model, such as an LLM, to perform the prediction. Formally, let $\mathcal{N}$ denote the set of input concepts. The relation prediction component produces a set of candidate relations $\mathcal{E}$ along with a weight mapping $w : \mathcal{E} \rightarrow \mathbb{R}$, where $w_{\vec{AB}}$ represents the weight assigned to the relation indicating that concept A is the parent of concept B.

b) *Post-processing:* Given a set of weighted edges, the post-processing stage aims to determine the most likely taxonomy based on these edges. Additionally, structure constraints may be taken into consideration during this process. Let $G = (\mathcal{N}, \mathcal{E})$ represent the weighted output graph of the relations predicted by the *relation prediction* component and let w be the corresponding weight mapping. Furthermore, let w_{max} denote the maximum possible weight that an edge can have. For all edges $\vec{AB} \in \mathcal{E}$, we define the *weight of a subgraph* $S = (\mathcal{N}, \mathcal{E}_S)$ of G as

$$W(S, G) = \sum_{\vec{AB} \in \mathcal{E}_S} w_{\vec{AB}} + \sum_{\vec{AB} \notin \mathcal{E}_S} (w_{max} - w_{\vec{AB}}) \quad (1)$$

Then, the task of identifying the most likely taxonomy can be formulated as a *constraint optimization* problem, aiming to find the *subgraph with the maximum weight* while adhering to all the specified *constraints*.

$$\max_S W(S, G) \text{ subject to } c \in \mathcal{C} \quad (2)$$

In this paper, we investigate both the above optimization problem, which considers the constraints, and a relaxed version of the problem that disregards these constraints.

C. Fine-tuning LLMs

Following fine-tuning for taxonomy construction, the LLM first assigns scores to *all pairs* of input concepts. These scores represent the likelihood of hierarchical relations among these pairs. Then, a taxonomy is derived based on the relation scores. The overview of this approach is illustrated in Figure 2(b).

a) *Relation prediction:* For the fine-tuning of LLMs, we adopt the pairwise classification commonly used by others [5]

Given a concepts pair (A, B) , we frame the task as a sentence classification problem aiming to assign a label to a sentence. We use the LLM to classify a sentence expressing the relation between concept A and B as true or false. We use the following template to create the input to the LLM: *I am doing the taxonomy research. I think B is a subtopic of A.* For example, in Figure 1, the input for *machine learning* and *supervised learning* would be expressed as *I am doing the taxonomy research. I think supervised learning is a subtopic of machine learning.* Although various templates are feasible, previous research indicates they have minimal impact on performance [5].

To construct a training set, we generate a collection of positive and negative examples based on a given ground-truth taxonomy. Considering that the number of negative examples can grow exponentially with the number of concepts, we employ *negative sampling* [15] to create negative examples.

- 1) For each edge $\vec{AB}$ in the taxonomy, we use the template to generate a positive example.
- 2) We randomly select N concept pairs (A, B) where B is not a descendant of A in the taxonomy.

Then, the LLM is fine-tuned to predict the correctness of the input sentence using the cross-entropy loss.

During inference, the *relation prediction* component assigns scores to *all pairs* of input concept as weighted relations.

b) *Post-processing:* Given a set of concepts for inference, we first use the fine-tuned LLM to compute scores for *all* concept pairs and subsequently identify the *most likely* taxonomy based on these pairwise scores. In this scenario, we explore two distinct variations of the post-processor that correspond to different forms of the optimization problem described in Equation 2: the *maximum-likelihood post-processor*, which disregards the constraints and the *maximum arborescence post-processor*, which incorporated the constraints.

Maximum-likelihood: The LLM is trained to predict pairwise relations between concepts while disregarding taxonomy constraints. The output of the LLM for all pairs of concepts can be interpreted as the weighted graph G , where the edge weights represent the logarithm of probability scores predicted by the LLM. Let $P(A, B)$ denote the LLM predicted probability score for concepts A and B . We define the weight of each edge $w_{\vec{AB}}$ as $w_{\vec{AB}} = \log(P(A, B))$

The weight of a subgraph in Equation 1 is determined by the sum of the logarithmic probabilities of edges in the subgraph using the Naive Bayes assumption. When disregarding the constraints, the subgraph with the maximum weight can be

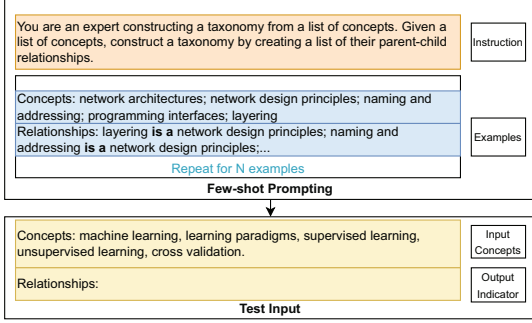


Fig. 3: An example of the few-shot prompt

obtained using the maximum likelihood approximation. For each edge $\vec{AB} \in \mathcal{E}$, include $\vec{AB}$ to S if $P(\vec{AB}) > 0.5$.

Maximum arborescence: Disregarding all constraints as the maximum likelihood approach can create inconsistent taxonomies that affect performance. An approach to address the optimization problem in Equation 2 involves using constraint optimization techniques, such as formulating the problem as MAXSAT and employing a constraint solver [14]. However, this approach can be computationally expensive and ignores the graph structure inherent in the taxonomy construction task.

Instead, we use the approach by Chen et al. [5]. The problem of optimizing the subgraph that adheres to all taxonomy constraints can be treated as a *maximum spanning arborescence* problem. A spanning arborescence is analogous to a spanning tree in directed graphs. We use the Edmonds algorithm [16] to identify the arborescence as the final output taxonomy, satisfying all imposed constraints for a tree-structured taxonomy.

D. Prompting with GPT-3.5

When using prompting with recent powerful LLMs such as GPT-3, accessed through APIs, the cost is typically determined by the API call count or tokens utilized in the request. Consequently, assigning scores to all pairs becomes highly expensive due to the quadratic growth in the number of pairs and, thus, a quadratic number of LLM calls. Alternatively, we introduce a more challenging task for the prompting setting, aiming to generate *all* candidate relations directly from the set of concepts with one LLM call. This is formulated as a *text generation* problem from concepts to relations. Figure 2(c) shows the overview of the prompting approach.

a) *Relation prediction:* To adapt the generative LLM to the taxonomy construction task, we use a few-shot prompt consisting of examples *randomly sampled* from the training set. An example of the few-shot prompt used in taxonomy construction is illustrated in Figure 3. The prompt provides the instruction for the taxonomy construction task, followed by a selection of concept-relation pairs (randomly sampled) from the training dataset. To mitigate any potential bias in the LLM regarding specific ordering, we randomly shuffle the order of concepts and relations in the examples. Finally, the prompt presents the input concepts in a shuffled order.

b) *Post-processing:* Due to the inherent randomness in beam search used by generative LLMs [8] and shuffle in the few-shot prompt, the outcome for generating taxonomy for the same concepts may vary across different runs. To address the issue of randomness, we propose a post-processing method that aggregates results from multiple runs through *majority voting*, motivated by the self-consistency prompting technique [17]. We disregard the taxonomy constraints in the prompting approach, deferring them to future works.

Using different few-shot examples, we obtain relation predictions for the same set of concepts N times. Then, the *relation aggregator* assigns a weight to each relation candidate based on their frequency of occurrence across the N generations. Denoting the count of occurrences of relation $\vec{AB}$ in the N generations as $c_{\vec{AB}}^N$, the weight of the relation is defined: $w_{\vec{AB}} = c_{\vec{AB}}^N$. Finally, the *majority voting post-processor* includes a relation into the final taxonomy if it appears in more than half of the generations: $w_{\vec{AB}} \geq \lceil \frac{N}{2} \rceil$.

IV. COMPUTER SCIENCE TAXONOMY DATASET

For this paper, we create a novel taxonomy construction dataset based on the Association for Computing Machinery (ACM) Computing Classification System (CCS) [18], an established ontology in the field of computer science.

CCS is a domain-specific ontology that includes the hierarchical relations for various terms from computing fields. We first convert the ontology in OWL format into a pairwise dataset. There are 13 taxonomies, each containing a number of terms ranging in [19, 323]. As shown in Figure 4, we apply a two-step data preprocessing method to transform the dataset into the standard format used in taxonomy construction [3].

Taxonomy split: The distribution of the number of nodes in the original taxonomies is skewed. The largest taxonomy consists of 323 concepts, which is 15 times larger than the smallest taxonomy, leading to a potential imbalance in the training process. To address this imbalance, following a similar approach to taxonomy processing in other domains [3], we split 13 taxonomies into 75 sub-taxonomies to achieve a more balanced distribution of terms. We split a taxonomy into sub-taxonomies if it has more than 70 terms. Specifically, the taxonomy is divided from the first level below its root (as depicted on the left side of Figure 4), resulting in all the direct children of the root becoming new roots in the sub-taxonomies.

Duplication removal: Following the split, we discover 144 duplicated relationships across the taxonomies. To prevent data leakage during training and achieve a more balanced distribution of the terms per taxonomy, we remove duplicates based on the following procedure.

- If some relations appear in multiple taxonomies, We keep only one instance of the duplicated relations in the taxonomy with the fewest terms among those taxonomies.
- However, if all relations in a taxonomy appear in other taxonomies, we remove that taxonomy from the dataset.

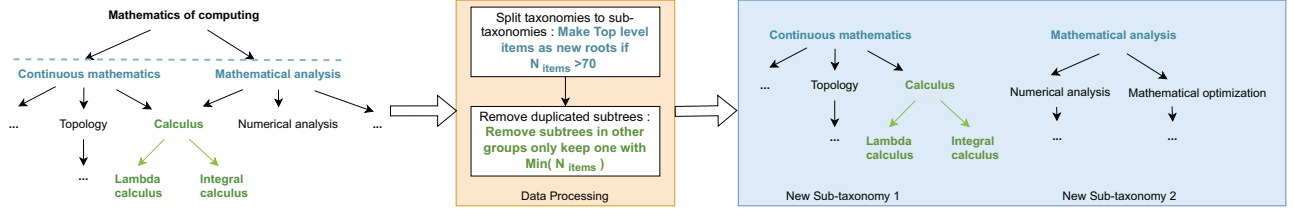


Fig. 4: Preprocessing of the ACM CCS dataset

V. EXPERIMENTS

In this section, we describe our experiment setup and evaluate the performance of the approaches. We compare two fine-tuning approaches with a few-shot prompting approach for LLMs. We evaluate the performance of these approaches in comparison to the ground truth and assess their consistency using the constraints. Furthermore, we discuss the implications of the experimental findings. Overall, the objective of this paper is to address the following research questions (RQs):

- 1) How do the two LLM-based approaches differ when compared to the ground truth?
- 2) What are the differences between the two LLM-based approaches in generating consistent taxonomies?

The implementation of approaches and the experiment setup can be found in the GitHub repository for this paper¹.

A. Setup

a) Dataset: We evaluate the approaches using two taxonomy datasets representing English language concepts and computer science concepts, respectively. Each dataset is divided into training (70%), validation (15%), and testing (15%), following the same splits as previous work [3], [5].

WordNet [6] is a hypernym taxonomy. We use a medium-sized variant of WordNet [3], which splits the original large taxonomy into several distinct subtrees. The dataset consists of 761 bottom-out non-overlapping taxonomies, containing 11 to 50 terms with a maximum tree height of three. The dataset contains 14,477 unique terms with 14,877 positive pairs.

ACM CCS is the novel taxonomy construction dataset from the ACM Computing Classification System as shown in Section IV. After the split, the dataset contains 75 non-overlapping taxonomies. While some taxonomies in the dataset contain nodes with multiple parents, deviating from the tree structure, we still apply the maximum arborescence post-processor for this dataset and leave the handling of such structures as future work.

b) Compared approaches: We compare two taxonomy construction approaches based on LLMs: fine-tuning and prompting, with multiple settings for each approach.

Fine-tuning: We perform fine-tuning on a GPT-NEO model [19] with 1.3 billion pre-trained parameters for language modeling on the Pile dataset [20]. Since we focus on comparing finetuning and prompting methods, to minimize the

impact of other factors such as model architecture, we omit masked language models like BERT with different model architectures than GPTs. We use two fine-tuning techniques for relation prediction: (1) **layer-wise**, where only parameters in the last few layers are updated during fine-tuning, and (2) **LoRA**, which updates all model parameters using low-rank decomposition. In the layer-wise approach, the parameters in the last five linear layers of the transformer block and the final linear layer are updated during fine-tuning. For LoRA, we set $d = 8$ and apply a LoRA dropout rate of 0.1.

In this setting, we use two post-processing approaches: (1) **MALI**: which is the *maximum likelihood* post-processor that selects the most probable subgraph from the relation predictions, and (2) **MSA**, which considers the taxonomy constraints and determines the *maximum spanning arborescence*.

Prompting: We access the **GPT-3.5-turbo** model through the OpenAI API². Specifically, we choose the snapshot of this model from June 13th, 2023. We select this model due to its balanced trade-off between inference performance and cost, making it suitable for comparison with the fine-tuning approaches. While more powerful models such as GPT-4 exist, their cost is over 20 higher than the GPT-3.5 models³, making them considerably more expensive than fine-tuning an LLM. We refer to this model as **GPT-3.5** throughout the experiment.

We conduct five generations using GPT-3.5 to generate candidate relations, aiming to mitigate the impact of randomness. For the WordNet dataset, we randomly select five taxonomies from the training dataset as few-shot examples. Since taxonomies from the CCS dataset are typically larger, we randomly select three as few-shot examples. Then, we compare two setups: (1) **IB**: The *individual best* performance among the five runs based on the evaluating metric and (2) **MV**: The taxonomy obtained through the *majority voting* post-processor.

B. RQ1 Taxonomy quality

a) Setup: This research question focuses on evaluating the generated taxonomies from different approaches in comparison to the ground truth taxonomies. Additionally, we include the state-of-the-art performance achieved on the WordNet dataset (**SOTA**). Since the CCS dataset is newly constructed, we establish a baseline by utilizing the performance of a randomly initialized LLM with the maximum arborescence post-processor (**Random**). Following prior research [3]–

¹<https://github.com/20001LastOrder/Taxonomy-GPT>

²<https://platform.openai.com/docs/models/gpt-3-5>

³<https://openai.com/pricing>

[5], we compare the generated and ground truth taxonomies by examining the ancestral relations. Concept A is considered an ancestor of concept B if there exists a path from A to B. We report the average precision, recall, and the F_1 score of the ancestral relations among all taxonomies.

b) *Results:* Table I shows the performance of all settings compared to the ground truth on the two datasets. We group the settings by the approach of relation prediction.

Overall, incorporating constraint-aware post-processor enhances the quality of the generated taxonomies by improving consistency. In the case of WordNet, while there is a marginal decrease in precision (approximately 5%) when comparing MSA and MALI across both layer-wise and LoRA settings, MSA has substantial improvement in recall and F_1 (around 10%) compared with MALI. Interestingly, there is a slight decrease in recall for MSA in case of CCS while improving in the precision and F_1 scores. This difference can be attributed to the domain and size distinctions between the two datasets. Similarly, the MV setting for GPT-3.5 also enhances precision and F_1 score for both datasets while sacrificing recall, compared with the best result for individual generation.

In comparison to the fine-tuning settings (layer-wise and LoRA), the prompting approach (GPT-3.5) yields better taxonomies. When considering the F_1 score, the best prompting setting (MV) outperforms the best fine-tuning setting on WordNet by 2.23%. In CCS, the difference becomes more significant, exceeding 20%. We attribute this gap in performance improvement to the smaller size of the CCS dataset, which makes the fine-tuning of an LLM more challenging.

Furthermore, even the best setting of GPT-3.5 falls short of the current state-of-the-art performance by 6.24% in F_1 , suggesting further enhancements through prompting and post-processing techniques. On the CCS dataset, all fine-tuning and prompting settings outperform the random baseline.

Answer to RQ1. The prompting method outperforms the fine-tuning method in both datasets when comparing the F_1 score. Moreover, the gap between the two methods increases when the training dataset is smaller.

C. RQ2 Taxonomy Consistency

a) *Setup:* This section focuses on quantifying the consistency of generated taxonomies. To assess the inconsistency level resulting from violations, we use the following four metrics. (1) **#Roots** represents the average number of roots per generated taxonomy. The ideal value should be one. (2) **NRG (%)** (No root graph) is the percentage of taxonomies without root. This scenario may arise due to cycles in the taxonomy, and a consistent taxonomy must always include a root. (3) **#Parents** indicates the average number of parents of all non-root nodes across the taxonomies. In a consistent taxonomy, each non-root node should have precisely one parent. (4) **MPN (%)** (Multi-parent nodes) is the average percentage of non-root nodes with multiple parents across the taxonomies. In a consistent taxonomy, the value for this

metric should be 0%. The first two metrics correspond to the **uniqueness of the root** constraint, and the latter two metrics are for the **uniqueness of the parent** constraint.

b) *Results:* As shown in Table II, all taxonomies generated by the fine-tuning methods (layer-wise and LoRA) with MSA are consistent with the imposed constraints. The MALI setting performs poorly for NRG, indicating the presence of cycles without root and leaf in certain taxonomies. Regarding the prompting method (GPT-3.5), IB and MV settings yield inconsistent taxonomies. However, the extent of inconsistency is much less compared to the MALI setting in fine-tuning.

When comparing the WordNet and CCS datasets, the taxonomies generated in the CCS dataset are more consistent than those in WordNet. These differences may originate from the difference in dataset domains. The CCS dataset specifically contains computer science terms, whereas WordNet represents a hypernym taxonomy with more general terms.

Answer to RQ2. The fine-tuning methods can produce taxonomies satisfying all constraints with the MSA post-processor. However, taxonomies generated by the prompting approaches still violate some constraints.

D. Discussion

This section explores various implications of the experimental findings and discusses the limitations of the experiments.

Approach selection: In RQ1, we observe that prompting a powerful LLM outperforms fine-tuning a weaker LLM. This performance gap becomes more significant when the training dataset is smaller. The scarcity of data is one factor hindering the adaptation of machine learning-based approaches to software engineering challenges. Our experimental finding demonstrates the potential of leveraging powerful LLMs through prompting techniques in addressing such challenges. We also note none of the approaches beat the current **SOTA** method based on BERT in F_1 score on WordNet. This suggests the model architecture may impact the task performance. The task may benefit from the bi-directional information flow in BERT compared to the left-to-right flow in GPTs.

However, Our experiments focus on two domains with publicly accessible datasets, which may be used as a part of the training data for the powerful LLM, resulting in good performance during evaluation. Besides, the setup assumes that concepts have semantically meaningful names, which might not be applicable to scenarios where concepts have ambiguous names or names heavily reliant on contextual interpretation. Future work will focus on overcoming these limitations.

Taxonomy consistency: In RQ1, we find that enhancing consistency through a constraint-aware post-processor improves the quality of the generated taxonomy in the fine-tuning approach. However, in RQ2, we discover that the majority voting post-processor of the prompting method still generates inconsistent taxonomies. This suggests one possible way to further improve the prompting performance by developing a constraint-aware post-processor tailored to this approach.

Settings	Precision (%)	Recall (%)	F1 (%)
Layer-wise (MSA)	62.91	50.07	54.58
Layer-wise (MALI)	67.66	39.08	42.29
LoRA (MSA)	62.57	51.21	55.03
LoRA (MALI)	66.98	42.73	46.97
GPT-3.5 (IB)	64.64	53.65	57.25
GPT-3.5 (MV)	77.50	47.98	57.26
SOTA [5]	67.30	62.00	63.50

TABLE I: Evaluation results for the WordNet (left) and CCS (right) dataset, averaged over all test taxonomies

Settings	Precision (%)	Recall (%)	F1 (%)
Layer-wise (MSA)	43.36	41.83	41.65
Layer-wise (MALI)	26.23	55.46	26.51
LoRA (MSA)	44.18	45.45	43.60
LoRA (MALI)	33.16	51.68	33.46
GPT-3.5 (IB)	66.23	63.18	61.07
GPT-3.5 (MV)	84.73	55.80	64.18
Random	16.21	17.44	16.07

Approaches	#Roots	NRG (%)	#Parents	MPN (%)
Layer-wise (MSA)	1.00	0.00	1.00	0.00
Layer-wise (MALI)	1.48	16.51	1.47	31.28
LoRA (MSA)	1.00	0.00	1.00	0.00
LoRA (MALI)	1.06	43.36	2.56	66.32
GPT-3.5 (IB)	2.00	4.39	1.00	0.04
GPT-3.5 (MV)	2.25	1.75	1.00	0.15

Approaches	#Roots	NRG (%)	#Parents	MPN (%)
Layer-wise (MSA)	1.00	0.00	1.00	0.00
Layer-wise (MALI)	0.57	64.29	3.30	59.81
LoRA (MSA)	1.00	0.00	1.00	0.00
LoRA (MALI)	1.36	28.57	2.60	65.13
GPT-3.5 (IB)	1.27	0.00	1.00	0.00
GPT-3.5 (MV)	2.87	0.00	1.01	0.78

TABLE II: Consistency evaluation for the WordNet (left) and CCS (right) dataset, averaged over all test taxonomies

We focus on two constraints that enforce the tree structure of taxonomy. However, some ground truth taxonomies in the CCS dataset violate this condition by having nodes with multiple parents. Additionally, other domains may impose other types of constraints on taxonomies. Therefore, one can explore methods for generating consistent taxonomies from both approaches while accounting for general constraints.

Additionally, the behavior of online LLMs, such as ChatGPT, may change due to internal updates of the model [21]. To avoid this change, we use the OpenAI API and choose a fixed model snapshot. Our approaches do not distinguish the definition of relations in the taxonomy. For example, while hypernym relations in WordNet and domain-specific relations in CCS both represent hierarchical relations, their definitions differ. One may encode this information to further improve the performance. We will leave this improvement as future work.

VI. RELATED WORK

Automated taxonomy construction: Most existing approaches use NLP-based methods automatically constructing hypernym taxonomies, such as WordNet. Bansal et. al. [3] first discuss the task of taxonomy construction using WordNet sub-trees and propose a method based on belief propagation. Mao et. al. [4] propose an end-to-end approach for taxonomy construction using reinforcement learning. Chen et. al. [5] present a taxonomy construction method by training a masked language model and outperforming previous methods. Recent studies have shown a growing interest in taxonomy construction using minimally supervised learning through the adaptation of pre-trained language models [22], [23].

LLMs for MDE: Despite the growing interest of LLMs such as GPTs [8] and BERT [24], their application in model-based software engineering (MDE) remains limited and has yet to achieve widespread adoption. Due to the sparsity of data, existing approaches typically involve fine-tuning using a small dataset, leading to modeling tools such as recommender systems for meta-modeling [25]. Still, the emergence of powerful generative LLMs like GPT4 [10] paves the way for novel

methods in adapting LLMs in MDE [26]. Specifically, there is a growing interest in using LLMs with prompting techniques. These approaches are often in an interactive context, such as using GPT-3 for model completion [27] or creating a chat-like experience with ChatGPT for UML modeling [28]

Compared to previous studies, we provide a systematic analysis of LLMs for taxonomy construction, using fine-tuning and prompting approaches. We also present a novel taxonomy construction dataset in the computer science domain. From a model-based perspective, we assess the consistency of generated taxonomies using structural constraints, while prior works emphasize performance compared to ground truth.

VII. CONCLUSION

This paper presents using LLMs in taxonomy construction, which involves creating hierarchical relations frequently used in software models. We conduct experiments on both a widely adopted hypernym dataset, along with a novel dataset derived from a computer science ontology. We incorporate the concept of *consistency*, a widely employed principle in model-based software modeling, into taxonomy construction. We demonstrate that enhancing consistency leads to an improvement in the quality of the taxonomy compared to the ground truth. We compare fine-tuning and prompting approaches and show that the prompting method outperforms fine-tuning, especially when the training dataset is small.

This work discovers the ability of LLMs for taxonomy construction and we hope it can serve as a potential guideline for integrating LLMs with other taxonomy construction methods to further improve the quality of resulting taxonomies. This improvement may involve (1) using complex prompting methods [29] to integrate other tools in the taxonomy construction process, (2) enhancing the performance of prompting methods through constraint-aware post-processing, (3) encoding domain-specific definition of the taxonomy relations, as well as (4) extending the comparison to other domains with various types of constraints.

REFERENCES

- [1] A. Cocos, M. Apidianaki, and C. Callison-Burch, "Comparing constraints for taxonomic organization," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018, pp. 323–333.
- [2] D. L. McGuinness, F. Van Harmelen *et al.*, "Owl web ontology language overview," *W3C recommendation*, vol. 10, no. 10, p. 2004, 2004.
- [3] M. Bansal, D. Burkett, G. De Melo, and D. Klein, "Structured learning for taxonomy induction with belief propagation," in *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2014, pp. 1041–1051.
- [4] Y. Mao, X. Ren, J. Shen, X. Gu, and J. Han, "End-to-end reinforcement learning for automatic taxonomy induction," *arXiv preprint arXiv:1805.04044*, 2018.
- [5] C. Chen, K. Lin, and D. Klein, "Constructing taxonomies from pre-trained language models," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. ACM, 2021.
- [6] G. A. Miller, "Wordnet: a lexical database for english," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [8] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [9] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.
- [10] OpenAI, "GPT-4 technical report," 2023.
- [11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*, 2022.
- [12] C. Li, H. Farkhoor, R. Liu, and J. Yosinski, "Measuring the intrinsic dimension of objective landscapes," *arXiv preprint arXiv:1804.08838*, 2018.
- [13] A. Aghajanyan, L. Zettlemoyer, and S. Gupta, "Intrinsic dimensionality explains the effectiveness of language model fine-tuning," *arXiv preprint arXiv:2012.13255*, 2020.
- [14] B. Chen, K. Marussy, S. Pilarski, O. Semeráth, and D. Varro, "Consistent scene graph generation by constraint optimization," in *37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.
- [15] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in neural information processing systems*, vol. 26, 2013.
- [16] J. Edmonds *et al.*, "Optimum branchings," *Journal of Research of the national Bureau of Standards B*, vol. 71, no. 4, pp. 233–240, 1967.
- [17] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.
- [18] "ACM computing classification system," Retrieved from <https://dl.acm.org/ccs>, New York, NY, 2023, accessed: 2023-06-21.
- [19] S. Black, G. Leo, P. Wang, C. Leahy, and S. Biderman, "GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow," Mar. 2021, If you use this software, please cite it using these metadata.
- [20] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima *et al.*, "The pile: An 800gb dataset of diverse text for language modeling," *arXiv preprint arXiv:2101.00027*, 2020.
- [21] L. Chen, M. Zaharia, and J. Zou, "How is chatgpt's behavior changing over time?" *arXiv preprint arXiv:2307.09009*, 2023.
- [22] D. Jain and L. E. Anke, "Distilling hypernymy relations from language models: On the effectiveness of zero-shot taxonomy induction," *arXiv preprint arXiv:2202.04876*, 2022.
- [23] L. Sun, Y. Shen, and W. Lu, "Minimally-supervised relation induction from pre-trained language model," in *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022, pp. 1776–1786.
- [24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [25] M. Weyssow, H. Sahraoui, and E. Syriani, "Recommending metamodel concepts during modeling activities with pre-trained language models," *Software and Systems Modeling*, vol. 21, no. 3, pp. 1071–1089, 2022.
- [26] B. Combemale, J. Gray, and B. Rumpe, "Chatgpt in software modeling," *Software and Systems Modeling*, vol. 22, 05 2023.
- [27] M. B. Chaaben, L. Burgueño, and H. Sahraoui, "Towards using few-shot prompt learning for automating model completion," in *IEEE/ACM International Conference on Software Engineering (ICSE 2023 NIER track)*, 2023, in press.
- [28] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative ai in modeling tasks: An experience report with chatgpt and uml," vol. 22, no. 3, p. 781–793, may 2023.
- [29] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.